

PE-EC702C — Neural Network & Fuzzy Logic Control

Semester VII • 3L:0T:0P • 3 Credits

Module 1 Exam Notes: Neural Networks and Pattern Association

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Biological vs. Artificial Neural Networks	2
1.1	Structural and Functional Mapping	2
2	Typical Architectures of ANNs	2
3	Common Activation Functions	3
4	McCulloch-Pitts Neuron Model	3
4.1	Mathematical Formulation	4
4.2	Linear Separability and the XOR Problem	4
5	Hebb Net Learning Rule	4
5.1	Hebb Net Training Algorithm	5
6	Perceptron Learning Algorithm	5
7	Adaline and Madaline Networks	5
7.1	Adaline (Adaptive Linear Neuron)	5
7.2	Madaline (Many Adalines)	6
8	Pattern Association Networks	6
8.1	Classification of Associative Memories	6
8.2	Training Rules for Pattern Association	6
9	Bidirectional Associative Memory (BAM)	7
10	Worked Examples	7

1. Biological vs. Artificial Neural Networks

An Artificial Neural Network (ANN) is an information-processing paradigm inspired by the structure and functional aspects of biological neural systems.

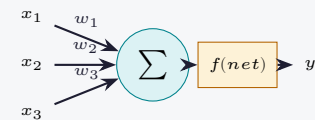
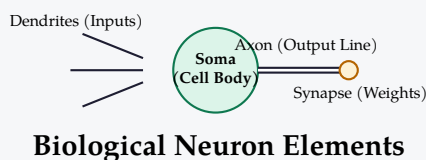
Artificial Neural Network (ANN)

An **Artificial Neural Network (ANN)** is a parallel, distributed computational model consisting of simple processing units (neurons) interconnected by weighted links, designed to learn patterns and relationships from data.

1.1 Structural and Functional Mapping

The biological brain consists of billions of highly interconnected cells called neurons. The artificial neuron models this biological mechanism:

Biological vs. Artificial Neuron Mapping



The physical correspondences between biological and artificial components are summarized below:

Biological Component	Artificial Equivalent	Functional Role
Dendrites	Inputs (x_i)	Receives signals from other neurons
Soma (Cell Body)	Summing Junction (Σ)	Aggregates all incoming weighted inputs
Axon	Output Transmission Line	Carries the output activation signal
Synapse	Weights (w_i)	Regulates the strength of connections
Synaptic Threshold	Activation Function (f)	Determines output state based on total input

2. Typical Architectures of ANNs

Neural network topologies are broadly classified into three categories based on the flow of signals and feedback loops:

1. **Single-Layer Feedforward Network:** Consists of an input layer of source nodes that connects directly to an output layer of neurons. There are no feedback loops and no hidden layers. (Example: Hebb net, Perceptron, Adaline).
2. **Multi-Layer Feedforward Network:** Contains one or more hidden layers of processing units between the input and output layers. This architecture enables the network to learn non-linear decision boundaries. (Example: Backpropagation MLP).
3. **Recurrent (Feedback) Network:** Contains at least one feedback loop where output signals are fed back as inputs to neurons in previous or current layers. (Example: Hopfield Network, BAM).

3. Common Activation Functions

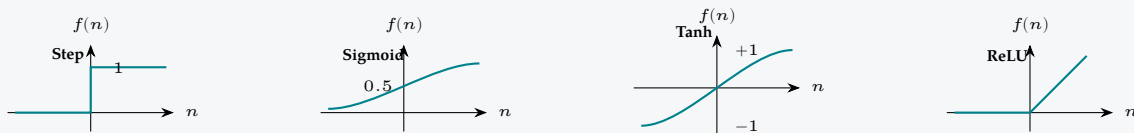
Activation functions (transfer functions) introduce non-linearity into the neuron model, allowing it to solve complex, non-linear problems.

Mathematical Formulations of Activation Functions

Let $net = \sum w_i x_i + b$ be the net input to the neuron.

Function	Mathematical Form	Range
Binary Step	$f(net) = \begin{cases} 1 & \text{if } net \geq 0 \\ 0 & \text{if } net < 0 \end{cases}$	$\{0, 1\}$
Bipolar Step	$f(net) = \begin{cases} 1 & \text{if } net \geq 0 \\ -1 & \text{if } net < 0 \end{cases}$	$\{-1, 1\}$
Binary Sigmoid	$f(net) = \frac{1}{1+e^{-\lambda net}}$	$(0, 1)$
Bipolar Sigmoid	$f(net) = \frac{2}{1+e^{-\lambda net}} - 1 = \tanh\left(\frac{\lambda net}{2}\right)$	$(-1, 1)$
Rectified Linear (ReLU)	$f(net) = \max(0, net)$	$[0, \infty)$

Activation Function Shapes



4. McCulloch-Pitts Neuron Model

The McCulloch-Pitts (M-P) neuron, introduced in 1943, was the first mathematical model of an artificial neuron.

McCulloch-Pitts Neuron

An **M-P Neuron** is a binary threshold device where inputs are either 0 or 1, weights are fixed integers (usually +1 for excitatory and -1 for inhibitory), and the output is determined by comparing the weighted sum against a threshold θ .

4.1 Mathematical Formulation

The activation of an M-P neuron is governed by:

$$y = f(net) = \begin{cases} 1 & \text{if } net \geq \theta \\ 0 & \text{if } net < \theta \end{cases} \quad (1)$$

where the net input net is computed as:

$$net = \sum_{i=1}^n w_i x_i \quad (2)$$

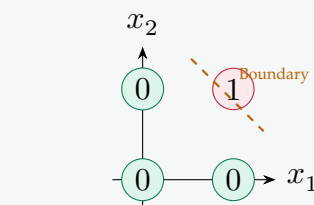
Inhibitory Rule: If any inhibitory input x_j is active (i.e. $x_j = 1$ where $w_j = -1$ or $-\infty$), the output y is forced to 0 immediately, regardless of the excitatory sum.

4.2 Linear Separability and the XOR Problem

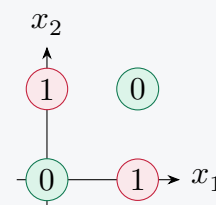
A classification problem is **linearly separable** if a single straight line (in 2D), plane (in 3D), or hyperplane (in higher dimensions) can completely separate the output classes.

Simple logic gates like AND, OR, and NOT are linearly separable. However, the XOR (Exclusive OR) gate is **not** linearly separable, as a single line cannot separate the outputs $\{0\}$ and $\{1\}$. This limitation led to the development of multi-layer neural networks.

Linear Separability of AND vs. XOR



AND Gate (Linearly Separable)



XOR Gate (Non-Linearly Separable)

5. Hebb Net Learning Rule

Proposed by Donald Hebb in 1949, this is the oldest learning algorithm. It is based on the biological premise: *"Neurons that fire together, wire together."*

Hebbian Learning Rule

If two connected neurons are active simultaneously, the weight of the connection between them should be increased:

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \quad (3)$$

where x_i is the input and y is the output.

5.1 Hebb Net Training Algorithm

1. Initialize all weights $w_i = 0$ and bias $b = 0$.
2. For each training input vector $s = (s_1, s_2, \dots, s_n)$ and target output t :
 - Set activation of input units $x_i = s_i$.
 - Set activation of output unit $y = t$.
 - Update weights: $w_i(\text{new}) = w_i(\text{old}) + x_i y$.
 - Update bias: $b(\text{new}) = b(\text{old}) + y$.

Note: Hebb net learning works best when inputs and targets are represented in bipolar form ($\{-1, 1\}$) rather than binary form ($\{0, 1\}$).

6. Perceptron Learning Algorithm

Developed by Frank Rosenblatt in 1958, the Perceptron is a supervised learning model used for binary classification.

Perceptron Activation & Weight Update

The output is computed using a binary/bipolar step activation:

$$y = \begin{cases} 1 & \text{if } \sum w_i x_i + b \geq \theta \\ -1/0 & \text{if } \sum w_i x_i + b < \theta \end{cases} \quad (4)$$

If the computed output y does not match the target t , the weights and bias are updated using the Perceptron Learning Rule:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y)x_i \quad (5)$$

$$b(\text{new}) = b(\text{old}) + \alpha(t - y) \quad (6)$$

where $\alpha \in (0, 1]$ is the learning rate.

7. Adaline and Madaline Networks

7.1 Adaline (Adaptive Linear Neuron)

Developed by Widrow and Hoff (1960), Adaline is a single-element network that utilizes the Delta Rule (LMS - Least Mean Squares) for training.

Adaline vs. Perceptron

Unlike the Perceptron where weights are updated based on the thresholded binary output, Adaline updates weights using the continuous analog output **before** the activation function is applied.

The weight update in Adaline minimizes the mean squared error (gradient descent):

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - \text{net})x_i \quad (7)$$

where $\text{net} = \sum w_i x_i + b$ is the analog pre-activation sum.

7.2 Madaline (Many Adalines)

Madaline consists of a multilayer organization of Adaline units. The input layer connects to a hidden layer of Adaline units, which in turn connect to Madaline output units using fixed logic functions (AND, OR, or Majority vote). Madaline is trained using MRI (Madaline Rule I) or MRII algorithms.

8. Pattern Association Networks

Pattern association involves storing set of pattern pairs (s^p, t^p) such that when an input pattern s^p is presented, the network recalls the associated target pattern t^p .

8.1 Classification of Associative Memories

1. **Auto-Associative Net:** A network where the input vector and target vector are identical ($s^p = t^p$). Its primary goal is pattern retrieval, denoising, or pattern completion. If a distorted or incomplete version of pattern s^p is presented, the network outputs the original stored pattern s^p .
2. **Hetero-Associative Net:** A network where the input vector and target vector are different ($s^p \neq t^p$, and they may reside in spaces of different dimensions). It maps an input vector in space X to an associated target vector in space Y .
3. **Iterative Auto-Associative Net:** A recurrent auto-associative network where output states are fed back as inputs dynamically. The network iteratively updates its activation values until it settles (converges) into a stable state representing one of the stored patterns (e.g., Hopfield networks).

8.2 Training Rules for Pattern Association

Training an associative memory network involves constructing a weight matrix W to map inputs to targets:

1. **Hebbian Learning Rule (Outer Product Rule):** For P bipolar pattern pairs (s^p, t^p) where $s^p \in \{-1, 1\}^n$ and $t^p \in \{-1, 1\}^m$, the weight matrix is formed by summing the outer products of input-target pairs:

$$W = \sum_{p=1}^P (s^p)^T t^p \quad \Rightarrow \quad w_{ij} = \sum_{p=1}^P s_i^p t_j^p \quad (8)$$

2. **Delta Rule (LMS / Widrow-Hoff Rule):** Used when patterns are not orthogonally separable. It is an iterative gradient descent rule that updates weights based on output error to minimize mean squared error:

$$\Delta w_{ij} = \alpha(t_j^p - y_j^p)s_i^p \quad (9)$$

where α is the learning rate, t_j^p is the target, and y_j^p is the actual output.

9. Bidirectional Associative Memory (BAM)

Introduced by Bart Kosko in 1988, BAM is a hetero-associative recurrent network that maps input vectors from space X to target vectors in space Y .

BAM Weight Matrix and Recall

Given P bipolar pattern pairs (S^p, T^p) where $S^p \in \{-1, 1\}^n$ and $T^p \in \{-1, 1\}^m$: The weight correlation matrix W of size $n \times m$ is computed as:

$$W = \sum_{p=1}^P (S^p)^T T^p \quad (10)$$

Bidirectional Recall Process:

- **Forward Direction:** Feed input vector X to recall output Y :

$$y_j = \text{sign} \left(\sum_{i=1}^n x_i w_{ij} \right) \rightarrow Y = \text{sign}(XW) \quad (11)$$

- **Backward Direction:** Feed output vector Y back to recall input X :

$$x_i = \text{sign} \left(\sum_{j=1}^m y_j w_{ij} \right) \rightarrow X = \text{sign}(YW^T) \quad (12)$$

10. Worked Examples

McCulloch-Pitts AND gate Implementation

Problem: Implement an AND gate using a McCulloch-Pitts neuron with binary inputs $x_1, x_2 \in \{0, 1\}$. Determine the weights and threshold θ .

Solution: Let the weights be $w_1 = 1, w_2 = 1$ (excitatory weights). The net input is $net = w_1 x_1 + w_2 x_2 = x_1 + x_2$. The output equation is $y = 1$ if $net \geq \theta$, else 0. Let's construct the truth table and formulate inequalities for θ :

x_1	x_2	Target t	$net = x_1 + x_2$	Condition for threshold θ
0	0	0	0	$0 < \theta$
1	0	0	1	$1 < \theta$
0	1	0	1	$1 < \theta$
1	1	1	2	$2 \geq \theta$

From the table, the threshold θ must satisfy:

$$1 < \theta \leq 2$$

Selecting $\theta = 2$ satisfies all conditions. Thus, the M-P neuron with weights $w_1 = 1$, $w_2 = 1$ and threshold $\theta = 2$ implements the AND gate.

Hebb Net Weight Calculation

Problem: Train a Hebb net for the AND gate using bipolar inputs and targets.

Solution: The training patterns are: $S^1 = (-1, -1), t^1 = -1$; $S^2 = (1, -1), t^2 = -1$; $S^3 = (-1, 1), t^3 = -1$; $S^4 = (1, 1), t^4 = 1$.

Initialize $w_1 = 0, w_2 = 0, b = 0$. The weight update rules are: $\Delta w_1 = x_1 y, \Delta w_2 = x_2 y, \Delta b = y$.

Step	x_1	x_2	y	Δw_1	Δw_2	w_1	w_2	b
Init	–	–	–	–	–	0	0	0
1	-1	-1	-1	1	1	1	1	-1
2	1	-1	-1	-1	1	0	2	-2
3	-1	1	-1	1	-1	1	1	-3
4	1	1	1	1	1	2	2	-2

The final weights are $w_1 = 2, w_2 = 2$ and bias $b = -2$. The decision boundary is: $2x_1 + 2x_2 - 2 = 0 \implies x_1 + x_2 = 1$.

BAM Correlation Matrix Computation

Problem: Store the bipolar hetero-associative pattern pair $S = (1, -1, 1)$ and $T = (1, -1)$ in a BAM. Calculate the weight matrix W and verify recall.

Solution: Given $S = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix}$ and $T = \begin{bmatrix} 1 & -1 \end{bmatrix}$.

The weight matrix is $W = S^T T$:

$$W = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$$

Verification of Forward Recall: Given input $S = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix}$:

$$SW = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 1(1) + (-1)(-1) + 1(1) & 1(-1) + (-1)(1) + 1(-1) \end{bmatrix}$$

$$SW = \begin{bmatrix} 3 & -3 \end{bmatrix}$$

Applying the sign function:

$$Y = \text{sign}(SW) = \begin{bmatrix} \text{sign}(3) & \text{sign}(-3) \end{bmatrix} = \begin{bmatrix} 1 & -1 \end{bmatrix} = T \quad (\text{Verified})$$

Quick Revision — Module I

Neural Networks and Pattern Association | PE-EC702C

Key Formulas at a Glance

M-P Neuron Output	$y = f(\sum w_i x_i) = \begin{cases} 1 & \text{if } \sum w_i x_i \geq \theta \\ 0 & \text{else} \end{cases}$
Hebbian Learning	$w_i(\text{new}) = w_i(\text{old}) + x_i y, \quad b(\text{new}) = b(\text{old}) + y$
Perceptron Learning Rule	$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y)x_i, \quad b(\text{new}) = b(\text{old}) + \alpha(t - y)$
Adaline (LMS) Delta Rule	$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - \text{net})x_i \quad \text{where } \text{net} = \sum w_i x_i + b$
BAM Weight Matrix	$W = \sum_{p=1}^P (S^p)^T T^p$
BAM Bidirectional Recall	$Y = \text{sign}(XW), \quad X = \text{sign}(YW^T)$

One-Line Definitions

- **Activation Function:** A mathematical function that determines the neuron's output state based on its net input.
- **Linear Separability:** The property where data classes can be separated by a single hyperplane.
- **Hebbian Learning:** A learning paradigm where connection weights increase when pre- and post-synaptic units are active together.
- **Adaline:** A single-layer neural network trained using gradient descent to minimize mean squared output error.
- **BAM:** A hetero-associative recurrent neural network that stores bipolar vector pairs and performs bidirectional recall.

Frequently Asked / Viva Questions

1. **Compare biological and artificial neural networks in terms of speed and processing style.**
 Biological networks process information in a massively parallel, slow ($\sim$ ms) style. ANNs process sequentially/quasi-parallelly but at extremely high electronic speeds ($\sim$ ns).
2. **Explain the XOR problem in single-layer perceptrons.**
 Single-layer perceptrons can only classify linearly separable patterns. XOR inputs cannot be separated by a single decision boundary line, rendering them unclassifiable by single-layer perceptrons.
3. **What is the significance of the bias term in a neuron?**

Frequently Asked / Viva Questions (contd.)

The bias acts as a shift factor for the activation function threshold, allowing the decision boundary to move away from the origin.

4. What is the difference between Perceptron learning and Adaline learning?

Perceptron learning updates weights based on the thresholded binary output, whereas Adaline learning uses the continuous pre-activation sum (Delta rule).

5. Define the stability-plasticity dilemma.

The dilemma of a network being stable enough to retain old learned patterns (stability) while remaining adaptive enough to learn new patterns (plasticity).

6. Why is bipolar representation preferred over binary in Hebb nets?

In binary representation (0, 1), inactive inputs (0) do not trigger weight updates, whereas bipolar representation (−1, 1) allows learning from negative states.

7. How does BAM retrieve stored patterns bidirectionally?

Inputs are multiplied by weight matrix W to yield outputs, which are then multiplied by transpose matrix W^T to retrieve inputs iteratively until stable convergence.

8. What is the difference between supervised and unsupervised learning?

Supervised learning uses target outputs to adjust weights based on error feedback. Unsupervised learning groups data patterns based on similarity metrics without targets.

9. What is the role of inhibitory inputs in the McCulloch-Pitts neuron?

Inhibitory inputs have absolute veto power: if an inhibitory input is active, the neuron output is forced to 0 regardless of excitatory inputs.

10. State the convergence theorem of perceptrons.

If the training data is linearly separable, the Perceptron learning algorithm is guaranteed to converge to a correct weight vector in a finite number of steps.

Comparison of Learning Rules

Parameter	Hebbian Learning	Perceptron Learning
Learning Type	Unsupervised (self-association)	Supervised (target-based)
Weight Update Condition	Always updates based on activity	Updates only on classification error
Convergence Guarantee	No guarantee of convergence	Guaranteed convergence if linearly separable
Feedback Loop	None	Error feedback loop present